

ALPHALIST DEVELOPER BOOTCAMP / HAMBURG / JUNE 16-17, 2026

Retrospective for your AI Factory

A retro for the factory you built yesterday: what happened, did it get better, did it stick.

Tereza Iofciu

IS THE CLIMB WORTH IT?

+30%

WITH A LICENSE ALONE

3-10x

WITH A FACTORY AROUND IT

Nobody shows you how these are measured. A single number called quality oversimplifies, and it means something different in every organization. Tomorrow morning you define what it means in yours.

BORROWED FROM AGILE

First, what is a retro?

IN SCRUM

Look back, then improve

A retrospective reviews the last sprint to decide what to keep, change, and act on. The sprint succeeds when every issue is closed.

WITH AGENTS

You manage the team now

A developer runs a team of agents. Same practice, pointed at the factory. But what counts as a good run is still up for debate.

In Scrum the finish line is given. For a factory of agents, you define it first.

EVALUATING THE FACTORY

Did this change make things better or worse?

If cost is the constraint

Better means fewer tokens for the same result.
Cheaper models per step, less context bloat.

If time is the constraint

Better means faster per delivered feature. Less wall-clock, less waiting on the loop.

If quality is the constraint

Better means fewer escaped defects and a lower change-failure rate. Correctness over speed.

If trust is the constraint

Better means more confidence per change: reviewable evidence, not a green checkmark.

You cannot maximize all four at once. Name the constraint you are under, then measure against it. That is tomorrow morning: data, not vibes.

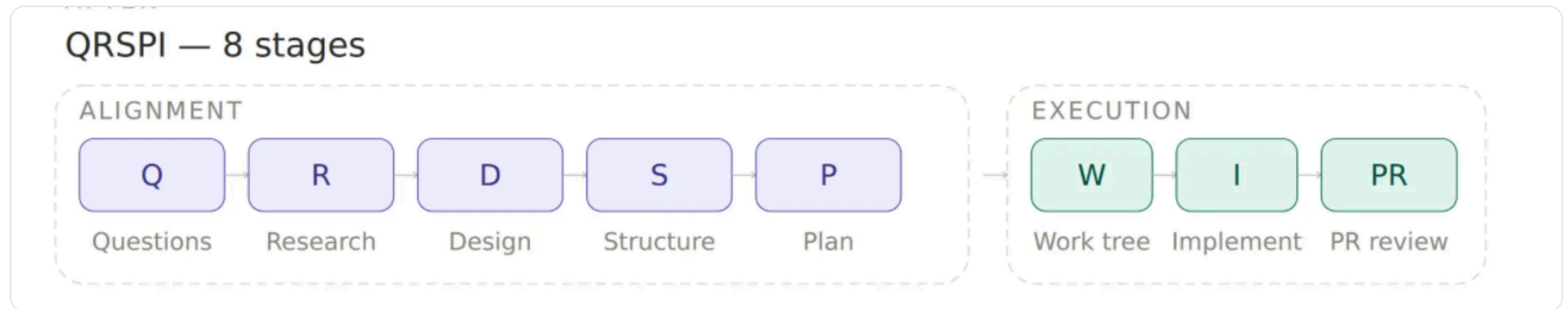
THE TWIST

Your factory is a **team**. Retro it.

A team retro produces action items for people. Here the action item changes the system itself, so the same miss cannot come back.

THE WHOLE LIFECYCLE

The factory is bigger than the code step.



Most failures happen before code. Eventually you want observability and eval at every agent in the loop.

How we want the system to run.

```
# Start work: feature branch, mark the issue in progress

# Develop (TDD loop)
#   1. failing test  2. make it pass  3. refactor  4. commit with the issue file

# Verify: a sub-agent checks it actually works

# Open the PR
git push -u origin <branch>
gh pr create --title "... " --body "... "

# Review (parallel sub-agents): post findings, fix the critical ones, wait for a human

# After merge: back to main, delete the branch, mark the issue done
```

THREE QUESTIONS, ONE LOOP

A retro asks three things.

What happened?

Did it stick?



Did it get better?

`Observability. Evals. Learning loops.`

Ask them at every stage, not just the code step.

TWO LOOPS, NOT ONE

The agent loops. You wrap it.



Inner loop: the agent. Outer loop: stage, artifact, review, budget.

Let the model run inside the loop. Do not let it be the loop.

LOOPS, THE PRODUCTION VERSION

17%

OF ORGS HAVE SHIPPED AGENTS

40%

OF AGENT PROJECTS CANCELLED BY 2027

The romantic version: write loops, a thousand agents build your company. The production version: write loops, and most of your job is making them halt. Gartner blames inadequate risk controls.

Make it halt. Three hard stops.

```
// the outer loop's stopping conditions
maxIterations: 8,
noProgressAfter: 2,          // consecutive steps with no new artifact
budgetCeilingUsd: 5.00,      // hard stop, no exceptions

// no-progress detection is "iterating too many times without converging",
// caught by the system instead of by you watching.
```

TWO LABS, ONE SPECTRUM

Monitor, or experiment.

CURSOR / WATCH PRODUCTION

Online signals

Keep Rate and accept rate from real sessions. Every prompt and harness change gets A/B tested against the live one.

ANTHROPIC / RUN EXPERIMENTS

Offline evals

Capability and regression suites. A broad per-model eval gates every change before it ships.

Production signals and a test suite. You need both ends.

ANTHROPIC, 2026

The model never changed. The **harness** did.

Six weeks chasing 'Claude got dumber.' A few quiet changes to the scaffolding caused it, not the model, and the eval suite was too narrow to see it. Watch the whole factory, not just the model.

DATA, NOT VIBES

Trace the run, not just the result.

Artifacts

the diff, the commits, the tests, the change log

MOST STOP HERE

Execution

the ordered tool calls: read, write, run, commit

THE PROCESS

Conversation

the transcript, the tokens spent

THE WHY

NOW YOU CAN ASK

A trace turns **vibes** into questions you can answer.

How many loops did it take? Is it converging, or thrashing? And the one that matters most: what does a good run even look like here?

SEE IT

Observability.

THIS WEEK

Read the record each task leaves

The plan it followed and the log of what it changed. Start with metrics you already track: lead time, change-failure rate, the DORA four.

IDEAL

Trace every step, on your own stack

What it read, ran, and retried. Point the observability you already run at your agents: Prometheus and Grafana, or a purpose-built tracer.

Track with the tools you already own. Do not adopt a toy.

Code your expectations.

```
# "Working" = these hold:  
1. never commit straight to main; always a feature branch  
2. never merge your own PR; a human reviews first  
3. tests before implementation  
4. smoke-test before the PR  
5. keep the issue checklist current  
  
# Every issue carries a Definition of Done.  
# You cannot mark it complete with unchecked items.  
  
# "Not working" = any one of these broke.
```

TEST IT

Evals.

THIS WEEK

One seeded defect, one gate

Plant a known mistake. A check catches it before code lands.
Most checks are a script: did it touch the right files, are the headings there, do the tests pass.

IDEAL

Scripts decide, a model judges the rest

A script for what rules can measure. A different model as judge for the quality they cannot, validated against human labels.

A 3 out of 5 hides the question. Pass or fail forces you to define good.

THE BRIDGE TO LOOPS

Every escape becomes a **test**.

A defect that slips past once is tagged with the PR that introduced it, then planted as a fixture. It can never slip past again. Public benchmarks rot and leak; your own escapes do not.

WHAT EVERYONE KEEPS LANDING ON

The same four rules, everywhere.

RULE 1

Failure cases and clean cases

Plant known defects, and include fixtures with none. Catch misses without crying wolf.

RULE 2

The judge is not the generator

Grade with a different model, or validate your judge against human labels.

RULE 3

Trends over absolutes

Gate on regressions and deltas, not a single green number.

RULE 4

Variance is a signal

Run it 3 to 5 times. The agent that passes once often fails on rerun. High spread means underspecified.

Anthropic's eval taxonomy, Husain and Shankar's field playbook, and two internal factories all keep landing here. Convergence is not coincidence. It is the shape of the problem.

WHEN BEHAVIOUR DIVERGES

Run the post-mortem to the system.

01 **Signal**

a rule broke, a metric moved, a human had to step in

02 **Diagnose**

why it happened, and where in the factory

03 **Change the system**

not the one artifact: the rule, the skill, the check

04 **Verify it holds**

re-measure against the baseline

05 **The next agent inherits it**

the fix lives where the next run will read it

Ask why until you hit the system.

01 "Claude got dumber"

the symptom everyone argued for six weeks

02 Why? Outputs regressed on our own tasks

measured, not felt

03 Why? A few tool definitions had changed

in the harness, not the model

04 Why? A scaffolding change shipped unreviewed

nothing gated it

05 Why? The evals watched the model, not the factory

the root cause

Stop at why one and you blame the model. Keep going and you fix the system. Five whys turns a postmortem into a change.

MAKE IT STICK

Improve by hand first.

FIRST, BY HAND

Fix it, then codify it

Turn one merged-PR learning into a rule the next agent reads. Reviews post findings; a rework pass fixes them; a repeated escape becomes a hook.

ONLY THEN, AUTOMATE

Wire the proven check into the loop

Once a check earns its keep by hand, make it a hook or a gate. Automation is the last step, not the first.

Land the change where the next run cannot miss it.

A working agreement no one checks is just a wish.

```
# expectations, enforced by hooks
```

```
PreToolUse hook    validates every branch name; a bad name is blocked.
```

```
PostToolUse hook   validates the Definition of Done; an issue with  
                    unchecked items cannot be marked complete.
```

```
# the rule that guides the edit is the rule the system checks.
```


TOOLING AT A GLANCE

Build it once by hand. Then choose a tool.

SEE IT

Tracing and metrics

OpenTelemetry as the wire format. Logfire or Langfuse purpose-built, or point Prometheus and Grafana, which you already run, at your agents.

TEST IT

Eval harnesses

pydantic-evals, Braintrust, Langfuse run your cases and track them over time.

Build the hook and the check by hand once, so you choose a tool instead of cargo-culting one. The principle outlives the vendor.

WHERE TO START

Start with the basic steps of your workflow.

01 **Observe**

trace one real run and see what actually happened

02 **Improve**

fix the one thing that broke, by hand

03 **Expand**

add the next stage, the next check

04 **Iterate**

run it again and let the small loop compound

A small loop on the workflow you already have. Start there.

WHO TUNES THE FACTORY?

Automation is the **last** step, not the first.

Stage by stage you hand more to agents. The meta layer, where you decide what to measure, what good means, and what to automate, stays yours. The Day 1 cases kept the human heavily in that loop. Alignment up front means the final review is a spot-check, not a slog.

How we'll know it worked.

```
# How we'll know it worked
```

```
Metric:      first-pass success (builds and tests, no human fix)
```

```
Baseline:    6 of 10 changes, today
```

```
The change:  add a plan check before code lands
```

```
Check-back:  re-measure after 10 changes, by 2026-07-15
```

```
Revert if:   it blocks a good change more than 1 in 10
```

```
# State all five, or the change fails review.
```

POD BLOCK 3, NEXT

What is quality for your factory?

Pick a metric

From a menu, or your own. Name the constraint it serves.

Measure it

Instrument it on the pipeline you built. Capture the baseline.

Improve one thing

Close one loop. One improvement end to end beats ten half-built.

The sandbox taught you the moves. Now run them on your factory. Two hours, and you leave with a number you did not have this morning.

START TODAY

Three tools you can open now.

THE STANDARD

OpenTelemetry

opentelemetry.io. Vendor-neutral traces and metrics. The format the rest can read.

PURPOSE-BUILT

Pydantic Logfire

pydantic.dev/logfire. Tracing made for LLM apps and agents: runs, tokens, and cost.

YOUR OWN STACK

Grafana

grafana.com. AI observability for agents, on the Prometheus and Grafana you already run.

One standard, one purpose-built tool, one you already own. Pick one and trace a single run this week.